

✓ NYAYA SETU (MVP) — Detailed Step-by-Step Build Plan (Tasks + Subtasks + Dependencies)

This plan is based on the final PRD and includes the **4 UI refinement goals**:

1. Navbar + Dark Mode polish
2. Premium homepage + cards
3. Results page better layout
4. Loading + error screens improvement

- ✓ Every task has dependencies
 - ✓ No orphan tasks
 - ✓ Each step is small + shippable
 - ✓ Follows best practices (incremental delivery, low risk jumps)
-

0) Project Initialization & Standards

✓ Task 0.1 — Repository + baseline tooling

Depends on: None

- Create git repo and push initial commit
 - Add .gitignore, .editorconfig, README.md
 - Configure TypeScript strict mode (or confirm it is enabled)
 - Add ESLint + Prettier config
 - Add scripts in package.json:
 - dev
 - build
 - lint
 - format
-

✓ Task 0.2 — Define shared types + constants (PRD contract)

Depends on: 0.1

- Create shared/constants.ts:
 - Allowed file types: PDF/JPG/PNG
 - Max upload size: 10MB
 - Supported languages (8)
 - Max stored docs: 3
 - Category buckets (5 + Other/Unclear)
 - Create shared/types.ts:
 - Language
 - UrgencyLevel (Low/Medium/High)
 - ExtractedFields (sender/receiver/date/deadline/subject/demands)
 - DecodedResult (docType, category, urgency, summary, keyPoints, actionPlan)
 - StoredDoc (id, title, createdAt, preview, decodedResult)
-

Task 0.3 — Route plan + flow rules (forward-only)

Depends on: 0.2

- Define official routes:
 - /login
 - /home
 - /upload
 - /preview
 - /language
 - /confirm
 - /loading
 - /result/:id
 - Write rules for accessing pages:
 - No auth → redirect to /login
 - No upload session → block preview/language/confirm/loading
 - No decode result → block result view
-

1) UI Foundation (Premium Baseline)

Task 1.1 — Tailwind base design system + layout wrapper

Depends on: 0.1

- Ensure TailwindCSS is installed and working

- Set darkMode: "class" in Tailwind config
 - Create common design helpers:
 - Card styling utility class
 - Badge styling utility class
 - PrimaryButton / SecondaryButton styling rules
 - Build AppShell component:
 - max width container
 - consistent padding
 - minimum height full screen
 - background support light/dark
-

✓ Task 1.2 — Add “Page template” component for consistent screens

Depends on: 1.1

- Create Page.tsx wrapper component:
 - title slot
 - optional description slot
 - children container
 - Apply Page wrapper to placeholder screens for visual consistency
-

2) Feature Set 1/4 — Navbar + Dark Mode Polish ✓

✓ Task 2.1 — Build Navbar component (minimal, premium)

Depends on: 1.2

- Create Navbar.tsx
 - Navbar must display:
 - App name: NYAYA SETU
 - Tagline: Legal clarity for everyone.
 - Right side area for actions (theme toggle, logout)
 - Keep minimal height, clean spacing, and consistent borders
-

✓ Task 2.2 — Implement Dark Mode toggle (manual)

Depends on: 2.1

- Create ThemeToggle.tsx
 - Store preference in localStorage
 - Toggle dark mode by adding/removing dark class on <html>
 - Ensure toggle exists only on /home (homepage-only rule)
-

✓ Task 2.3 — Verify dark mode compatibility across core UI elements

Depends on: 2.2

- Ensure readable contrast for:
 - navbar
 - cards
 - badges
 - buttons
 - Fix any illegible text in dark mode
-

3) Authentication (Mandatory OTP)

✓ Task 3.1 — Setup OTP provider config (Firebase recommended)

Depends on: 0.1

- Create Firebase project
 - Enable Phone OTP authentication
 - Add .env values (keys/config)
 - Implement Firebase init utility (firebase.ts)
 - Add error-safe config loading
-

✓ Task 3.2 — Build Login screen (Phone → OTP → Verify)

Depends on: 3.1, 1.2

- Create /login UI:
 - phone input
 - send OTP button
 - OTP input
 - verify button

- Handle auth errors cleanly (toast/message)
 - Redirect to /home on success
-

✓ Task 3.3 — Session + logout + auth guard

Depends on: 3.2, 0.3

- Create useAuth() hook:
 - current user
 - loading state
 - Protect routes:
 - unauthenticated → /login
 - Add logout action in Navbar (visible when logged in)
 - Ensure session persists ~30 days
-

4) Local Persistence (IndexedDB) + “Latest 3 Only”

✓ Task 4.1 — Implement IndexedDB database layer (Dexie)

Depends on: 0.2

- Install Dexie
 - Create db.ts with tables:
 - documents
 - Implement DB methods:
 - saveDocument(doc)
 - getDocumentById(id)
 - listRecentDocuments(limit=3)
 - deleteDocumentById(id)
-

✓ Task 4.2 — Enforce “keep only last 3 docs” rule

Depends on: 4.1

- Implement enforceMaxDocuments(3)
- On every new save:
 - if count > 3 → delete oldest
- Add dev logs to confirm deletion happens correctly

5) Feature Set 2/4 — Homepage Premium + Cards



✓ Task 5.1 — Build Homepage screen layout

Depends on: 2.3, 3.3

- Create /home screen with:
 - Navbar (with theme toggle)
 - Primary CTA: **Scan & Decode**
 - Recent documents section placeholder
- Hook CTA to /upload

✓ Task 5.2 — Premium Recent Docs cards (max 3)

Depends on: 4.2, 5.1

- Fetch recent docs from IndexedDB (limit 3)
- Render card per document:
 - title
 - created date
 - detected type badge (if available)
- Clicking card opens /result/:id

✓ Task 5.3 — Homepage empty state UX

Depends on: 5.2

- If no docs:
 - show helpful message (“Upload a document to begin”)
 - show CTA prominently

6) Upload Screen (PDF + Images)

✓ Task 6.1 — Build Upload screen UI + validation

Depends on: 5.1, 0.2

- Create /upload screen with:
 - upload PDF button
 - upload images button (multi-select)
 - Validate:
 - allowed types only
 - total size $\leq$ 10MB
 - Show error messages on invalid input
-

✓ Task 6.2 — Create decode session state store

Depends on: 6.1

- Implement useDecodeSession() store:
 - selected files
 - file type (pdf/images)
 - preview edits
 - selected language
 - decode status
 - Persist only in memory (session-level)
 - After upload validated → go to /preview
-

7) Preview + Crop/Rotate

✓ Task 7.1 — Build Preview screen

Depends on: 6.2

- Create /preview screen:
 - display preview of uploaded document
 - show crop and rotate controls
 - Add Continue button → /language
-

✓ Task 7.2 — Save crop/rotate edits into decode session

Depends on: 7.1

- Apply crop/rotate transformations to stored images

- Store final edited version in session state
 - Ensure edited version is used for decoding
-

8) Language Selection (Every Decode)

✓ Task 8.1 — Build Language selection screen

Depends on: 7.2, 0.2

- Create /language screen:
 - dropdown of 8 languages
 - selection required
 - Save selected language into decode session state
 - Continue → /confirm
-

9) Confirm Upload + Warning

✓ Task 9.1 — Build Confirm screen + keep-last-3 warning

Depends on: 8.1

- Create /confirm screen showing:
 - file name(s)
 - selected language
 - Show warning message:
 - “Only your latest 3 documents are kept.”
 - Proceed button → /loading
-

10) Feature Set 4/4 — Loading + Error Screens Improvement ✓

✓ Task 10.1 — Build Loading screen polish

Depends on: 9.1

- Create /loading screen:

- premium centered spinner
 - “Decoding...” text
 - Ensure UI doesn’t freeze during processing
-

✓ Task 10.2 — Build Decode failure UI + scan tips

Depends on: 10.1

- If decode fails:
 - show “Could not decode reliably”
 - show “Re-upload” button → /upload
 - show scan tips list (lighting, blur, crop, shadows)
-

11) OCR + Decode API Pipeline

✓ Task 11.1 — Implement OCR module (OCR everything)

Depends on: 9.1

- Create OCR wrapper:
 - PDF → OCR extraction (even if selectable text exists)
 - Images → OCR extraction
 - Return extracted text as a single string
 - Add timeouts and max text length cap
-

✓ Task 11.2 — Implement LLM decoder (strict output schema)

Depends on: 11.1, 0.2

- Create decoder prompt to output:
 - detected doc type
 - category tag
 - urgency label
 - extracted details fields
 - Summary (5–7 lines)
 - Key Points (6–10 bullets)
 - Action Plan = “Consult a lawyer.”
- Validate output schema (reject malformed output)

✓ Task 11.3 — Build

/api/decode

endpoint

Depends on: 11.2

- Accept payload:
 - uploaded file(s)
 - selected language
- Execute:
 - OCR → text
 - LLM decode → structured output
- Return JSON result to frontend

✓ Task 11.4 — Connect Loading screen to decode endpoint

Depends on: 11.3, 10.1, 10.2

- On /loading mount:
 - call /api/decode
 - success → store decoded output
 - failure → show failure UI + scan tips

12) Save Decoded Document Locally (and enforce latest 3)

✓ Task 12.1 — Save decoded doc into IndexedDB

Depends on: 11.4, 4.2

- Create StoredDoc object:
 - id
 - createdAt
 - default title
 - preview reference (first page)
 - decoded result

- extracted raw OCR text
 - Save to IndexedDB
 - Enforce keep-only-last-3 rule
 - Redirect to /result/:id
-

13) Feature Set 3/4 — Results Page Better Layout



✓ Task 13.1 — Results screen structure + warning banner

Depends on: 12.1

- Create /result/:id
 - Load stored document from IndexedDB
 - Show top banner:
 - “This may be inaccurate.”
-

✓ Task 13.2 — Add premium “Details” card + urgency label

Depends on: 13.1

- Show extracted fields at top:
 - sender, receiver, date, deadline, subject, demands
 - Show “Reply by: ”
 - Show urgency badge (Low/Medium/High)
 - Show detected doc type + category tag badges
-

✓ Task 13.3 — Tabs UI: Summary / Key Points / Action Plan

Depends on: 13.2

- Add tabs:
 - Summary (5–7 lines)
 - Key Points (6–10 bullets)
 - Action Plan: “Consult a lawyer.”
-

✓ Task 13.4 — Document preview (first page) + modal

Depends on: 13.2

- Render preview thumbnail
 - On click open modal viewer
 - Show only first page
-

✓ Task 13.5 — Collapsible “Original Extracted Text”

Depends on: 13.3

- Add collapsible section below tabs
 - Render OCR text read-only
 - Do not add copy buttons
-

14) Rename + Delete + Undo (5 seconds)

✓ Task 14.1 — Rename document

Depends on: 13.1

- Add rename button in results page
 - Modal input for title
 - Persist title change to IndexedDB
 - Ensure homepage recent cards reflect new title
-

✓ Task 14.2 — Delete document with confirm + undo toast

Depends on: 14.1

- Add delete button
- Confirmation modal required
- On delete:
 - remove from IndexedDB
 - show Undo toast (5 seconds)
- If Undo clicked:
 - restore document
- After final delete:

- redirect to /home
-

15) Export PDF (Single report)

✓ Task 15.1 — Implement Export PDF report

Depends on: 13.4, 13.3

- Add export button on results page
 - Generate single PDF including:
 - document preview (first page)
 - extracted details fields
 - Summary + Key Points + Action Plan
 - Ensure:
 - no watermark
 - normal PDF download
 - no disclaimer inside PDF
-

16) Rating + Feedback (KPI)

✓ Task 16.1 — Add rating widget + optional text feedback

Depends on: 13.3

- Place rating section on results screen
 - Allow 1–5 selection
 - Optional feedback text
 - Save feedback locally (IndexedDB) or POST to endpoint
-

17) QA + Demo Hardening

✓ Task 17.1 — End-to-end QA checklist run

Depends on: 16.1, 15.1, 14.2, 10.2

- Login works (OTP)
- Upload PDF works

- Upload images works
 - Preview crop/rotate works
 - Language selection works (8 languages)
 - Decode flow works (up to 60 seconds)
 - Results show proper tabs, details, preview, extracted text
 - Save-only-last-3 rule works (4th deletes oldest)
 - Rename works
 - Delete + Undo works
 - Export PDF works
 - Failure screen works + scan tips show properly
 - Dark mode works and looks premium
-

18) Deployment

Task 18.1 — Deploy + verify production stability

Depends on: 17.1

- Add production env vars (auth + OCR + LLM)
 - Deploy to hosting (Replit publish / Vercel)
 - Run production smoke test:
 - login
 - decode
 - export PDF
 - open recent docs
-

Dependency Graph (Simple Summary)

- Foundation: **0.1 → 0.2 → 0.3**
- UI Base: **1.1 → 1.2**
- Navbar/Dark Mode: **2.1 → 2.2 → 2.3**
- Auth: **3.1 → 3.2 → 3.3**
- Storage: **4.1 → 4.2**
- Homepage: **5.1 → 5.2 → 5.3**
- Upload Flow: **6.1 → 6.2 → 7.1 → 7.2 → 8.1 → 9.1**
- Loading/Error: **10.1 → 10.2**

- Decode Pipeline: **11.1** → **11.2** → **11.3** → **11.4**
- Save Decoded: **12.1**
- Results Polish: **13.1** → **13.2** → **13.3** → **13.4** → **13.5**
- Actions: **14.1** → **14.2**
- Export: **15.1**
- KPI: **16.1**
- QA + Deploy: **17.1** → **18.1**
 -